System Architecture & Design Patterns

Building resilient multi-component systems

SEAtS | 255092

University of Southampton

Intended Learning Outcomes

By the end of this lecture, you will be able to:

1. Explain the role of system architecture in structuring multi-component solutions.
2. Compare decomposition strategies and assess their trade-offs.
3. Recognise key software, hardware, and cross-domain design patterns and when to apply them.
4. Integrate stakeholder analysis to inform architectural decisions.
5. Evaluate architecture choices using optimisation principles and real-world constraints.

© University of Southampton

Agenda

- Why Architecture Matters
- Systems Thinking Lens
- Architectural Views & Styles
- Decomposition & Coordination
- Design Pattern Toolbox
- Stakeholders & Requirements
- Optimisation & Trade-offs
- Case Study: Smart Home IoT Control
- Summary

© University of Southampton

Why Architecture Matters

System architecture defines how components collaborate, how responsibilities are separated, and how a solution adapts to change. Without a coherent architecture, teams encounter integration surprises, runaway complexity, and brittle designs.

⚠ Common failure modes

- Treating multi-disciplinary systems as monoliths → coupling and rework.
- Missing coordination plans between hardware, firmware, and software teams.
- Deploying without visibility of dependencies or rollback strategies.

© University of Southampton

Systems Thinking Lens

Systems thinking anchors architecture decisions in the broader socio-technical context.

- Map interdependencies between subsystems, interfaces, and supporting services.
- Identify reinforcing and balancing feedback loops (e.g., telemetry improving release cadence vs. alert fatigue).
- Surface multiple perspectives: engineering, operations, end-users, regulators, supply partners.

ⓘ Systemic design mindset

- Work across disciplines early.
- Combine top-down (vision, goals) with bottom-up (constraints, local knowledge).
- Act locally while keeping global objectives visible.

© University of Southampton

The 4+1 View Model

```
graph TD
    SA[System Architecture]
    SA --> LV["Logical View<br/>What functions?"]
    SA --> PRV["Process View<br/>How it executes?"]
    SA --> DV["Development View<br/>How it evolves?"]
    SA --> PV["Physical View<br/>Where it runs?"]

    LV --> LVD[Class / Object diagrams]
    PRV --> PRVD[Sequence / Activity diagrams]
    DV --> DVD[Component / Package diagrams]
    PV --> PVD[Deployment diagrams]

    SV(["Scenarios +1<br/>Does it hang together?"]) --> SVD[Use case diagrams]
    SV -.-> LV
    SV -.-> PRV
    SV -.-> DV
    SV -.-> PV
```


© University of Southampton

A Diagram Type for Each View

Each view is documented with the UML (or UML-like) diagram best suited to the question it answers — Kruchten's "+1" is the **Scenarios view**: a handful of use cases that cut across all four, used to check they're actually consistent with each other.

View	Answers	Typical diagram(s)
Logical	What functions?	Class diagrams, communication/object diagrams
Process	How it executes?	Sequence diagrams, activity diagrams (concurrency, timing)
Development	How it evolves?	Component diagrams, package diagrams
Physical	Where it runs?	Deployment diagrams
Scenarios (+1)	Does it all hang together?	Use case diagrams

© University of Southampton

Logical View — Class & Object Diagrams

- **Answers**: what functions and responsibilities exist, and how they relate.
- Captures modules, their data, and their associations — independent of runtime behaviour or deployment.
- **Class diagrams** show static structure; **object/communication diagrams** show a snapshot of specific instances collaborating.

```
classDiagram
    class SecureHub
    class LightingController
    class HVACUnit
    class DoorLock
    class DeviceStateTwin
    SecureHub --> LightingController : controls
    SecureHub --> HVACUnit : controls
    SecureHub --> DoorLock : controls
    SecureHub --> DeviceStateTwin : reports state
```


© University of Southampton

Process View — Sequence & Activity Diagrams

- **Answers**: how the system executes — concurrency, timing, synchronisation between runtime processes.
- **Sequence diagrams** order the messages exchanged between participants; **activity diagrams** show branching/parallel control flow.
- Same modules as the Logical View, now shown interacting over time.

```
sequenceDiagram
    participant App as Mobile App
    participant Broker as Command Broker
    participant Hub as Secure Hub
    participant Light as Lighting Controller
    App->>Broker: Turn on lights
    Broker->>Hub: Dispatch command
    Hub->>Light: Set state = ON
    Light-->>Hub: Ack
    Hub-->>Broker: State updated
```


© University of Southampton

Development View — Component & Package Diagrams

- **Answers**: how the system evolves — the programmer's-eye view of modules, libraries, and build dependencies.
- **Component diagrams** expose interfaces between deployable units; **package diagrams** group related code and show compile/build-time dependencies.
- Organised by repository, not by runtime behaviour.

```
graph TD
    subgraph Repo1["repo: mobile-app"]
        UIModule[UI Components]
        AuthModule[Auth Client]
    end
    subgraph Repo2["repo: cloud-services"]
        BrokerPkg[Command Broker]
        TwinPkg[Device Twin]
    end
    subgraph Repo3["repo: hub-firmware"]
        CommsPkg[Comms Stack]
        DriverPkg[Device Drivers]
    end
    UIModule --> AuthModule --> BrokerPkg
    BrokerPkg --> TwinPkg
    BrokerPkg --> CommsPkg --> DriverPkg
```


© University of Southampton

Physical View — Deployment Diagrams

- **Answers**: where the system runs — software mapped onto hardware, nodes, and network topology.
- **Deployment diagrams** show processors, devices, and the protocols connecting them.
- Surfaces latency, bandwidth, and failure-domain concerns the other views hide.

```
graph TB
    subgraph CloudRegion["Cloud Region"]
        BrokerNode["Broker VM"]
        TwinDB["Twin Database"]
    end
    subgraph HomeLAN["Home LAN"]
        HubDevice["Secure Hub (Raspberry Pi)"]
        LightNode["Lighting MCU"]
        HVACNode["HVAC MCU"]
    end
    Phone["Mobile Phone"] -- "Wi-Fi/4G" --> BrokerNode
    BrokerNode -- "MQTT/TLS" --> HubDevice
    HubDevice -- "Zigbee" --> LightNode
    HubDevice -- "Zigbee" --> HVACNode
    BrokerNode --- TwinDB
```


© University of Southampton

Scenarios (+1) View — Use Case Diagrams

- **Answers**: does it all hang together? Not a separate structure — a handful of significant use cases that exercise the other four views at once.
- **Use case diagrams** show actors and the goals they want the system to achieve, linked by include/extend relationships.
- Used to sanity-check that the Logical, Process, Development, and Physical views are consistent with each other.

```
flowchart LR
    Homeowner((Homeowner))
    Guest((Guest))
    Installer((Installer))
    Homeowner --- UC1([Turn on lights remotely])
    Homeowner --- UC2([Adjust HVAC schedule])
    Guest --- UC3([Request temporary access])
    Installer --- UC4([Onboard new device])
    UC1 -.includes.-> UC5([Authenticate user])
    UC2 -.includes.-> UC5
```


© University of Southampton

Architectural Styles

✓ Layered

- Presentation → business → data layers.
- Example: web stack with frontend UI, API services, databases.

ⓘ Service-Oriented

- Loosely coupled services via contracts.
- Example: payment service, identity service, inventory service.

⚠ Event-Driven

- Publish/subscribe with async processing.
- Example: IoT sensors emitting events consumed by analytics pipeline.

✓ Component-Based

- Reusable modules with defined interfaces.
- Example: modular robotics joints or PCB daughterboards.

© University of Southampton

Decomposition and Coordination

Breaking a complex system into manageable parts demands a coordination plan.

ⓘ Horizontal decomposition

- Split peers at the same abstraction: e.g., mechanical arm segments, microservices, power subsystems.
- Trade-off: easier parallel development, but requires integration standards.

✓ Vertical decomposition

- Separate abstraction layers: system-level goals vs. component implementations.
- Enables focused optimisation at each level while preserving overall intent.

⚠ Informed coordination

- Use surrogate/meta models to communicate constraints (weight, latency, tolerances) between teams without over-coordination.
- Balance autonomy with shared metrics to prevent local optima from harming global outcomes.

© University of Southampton

Design Pattern Toolbox

Software Patterns

✓ Creational

- **Factory**: centralise object creation when variants exist.
  - Example: UI widget factory choosing native controls for macOS vs. Windows.
- **Singleton**: single instance for shared resources (e.g., configuration service).
  - Example: global telemetry collector wiring every subsystem to the same exporter.

© University of Southampton

Software Patterns

Structural

- **Adapter**: bridge incompatible interfaces (e.g., legacy API to new service).
  - Example: adapter translating CAN bus packets into the cloud telemetry schema.
- **Composite**: treat individual and grouped components uniformly (e.g., scene graph).
  - Example: robot subsystem tree where joints and limb assemblies expose the same commands.

© University of Southampton

Software Patterns

⚠ Behavioural

- **Observer**: notify subscribers about state changes (telemetry, UI bindings).
  - Example: dashboard widgets auto-refreshing when the process historian publishes updates.
- **Strategy**: swap algorithms without altering caller (e.g., control loop modes).
  - Example: autonomous vehicle selecting between eco, sport, or safety-first driving profiles.

© University of Southampton

Hardware / Embedded Patterns

- **Bus Controller**: manage shared communication channels across peripherals.
  - Example: I2C master arbitrating sensor access on an embedded board.
- **Pipeline**: staged signal processing (sensor → filter → actuator).
  - Example: radar front-end → DSP filter → actuator control for adaptive cruise.
- **Redundancy**: duplicate components for failover (dual power supplies, redundant sensors).
  - Example: dual inertial measurement units with voting logic for aviation guidance.

© University of Southampton

Cross-domain Patterns

ⓘ Model-View-Controller

- Separates interaction (view), state (model), and coordination (controller).
- Works for embedded HMIs or web applications.
  - Example: factory touchscreen HMI (view) driving PLC state (model) via a controller layer.

⚠ Publisher-Subscriber

- Decouple producers and consumers via topics/events.
- Fits both software events and hardware interrupt signalling.
  - Example: robotics middleware broadcasting joint telemetry to both UI dashboards and fault detectors.

© University of Southampton

Stakeholders and Requirements

Effective architecture aligns with stakeholder needs beyond pure functionality.

ⓘ Stakeholder checklist

- Identify early: operators, maintainers, users, regulators, suppliers.
- Map influence vs. interest to prioritise engagement.
- Elicit both functional and non-functional needs (safety, maintainability, auditability).

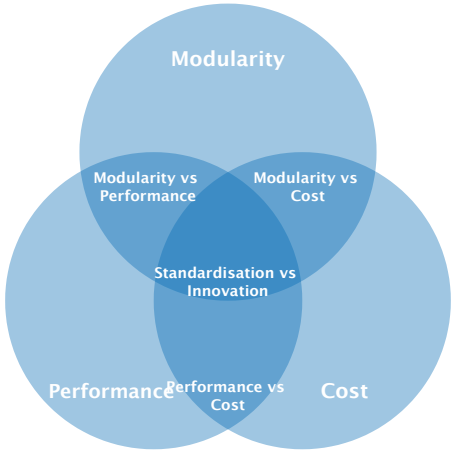
✓ Systemic inquiry tools

- Boundary scoping: what's inside vs. outside the system.
- Visual mapping: causal loops, journey maps, stakeholder grids.
- Ethnography: observe real usage and workflows.

© University of Southampton

Optimisation & Trade-offs

Architecture is a balancing act across competing objectives.



⚠ Optimisation tactics

- Use meta-modelling to evaluate candidate architectures quickly.
- Apply constraint handling to enforce weight, latency, cost ceilings.
- Iterate with real telemetry to refine assumptions.

© University of Southampton

Case Study: Smart Home IoT Control

✓ Architecture

- Split the solution into user control apps, cloud orchestration, and in-home hub plus devices.
- Use lightweight messaging (MQTT/WebSocket) for near-real-time commands and telemetry.
- Offer local fallbacks and role-based access so safety-critical devices stay controllable during outages.

⚠ Outcome

- Achieved sub-second command latency for 90% of lighting and HVAC actions.
- Cut support tickets by 35% through proactive device health telemetry.
- Architecture lets partners onboard new device types via the shared digital twin schema.

© University of Southampton

Smart Home IoT Control — Architecture

```
graph TD
    subgraph User["User Control"]
        App[Mobile App]
        Voice[Voice Assistant]
    end
    subgraph Cloud["Cloud Orchestration"]
        Broker[Command Broker]
        Twin[Device State Twin]
        Auth["Identity & Permissions"]
    end
    subgraph Home["Home Network"]
        Hub[Secure Hub]
        Lights[Lighting Controllers]
        HVAC[HVAC Unit]
        Locks[Door Locks]
    end

    App --> Auth
    Voice --> Auth
    App --> Broker
    Broker --> Twin
    Broker --> Hub
    Hub --> Lights
    Hub --> HVAC
    Hub --> Locks
    Hub --> Broker
    Hub --> Twin
    Twin --> App
    Twin --> Voice
```


© University of Southampton



© University of Southampton

Summary

- Architecture orchestrates people, processes, and technology for resilient delivery.
- Systems thinking keeps interdependencies, feedback loops, and stakeholder needs visible.
- Design patterns accelerate implementation while preserving flexibility.
- Optimisation techniques provide evidence for trade-offs under real constraints.
- Modular thinking plus informed coordination unlock scalable, maintainable solutions.

© University of Southampton